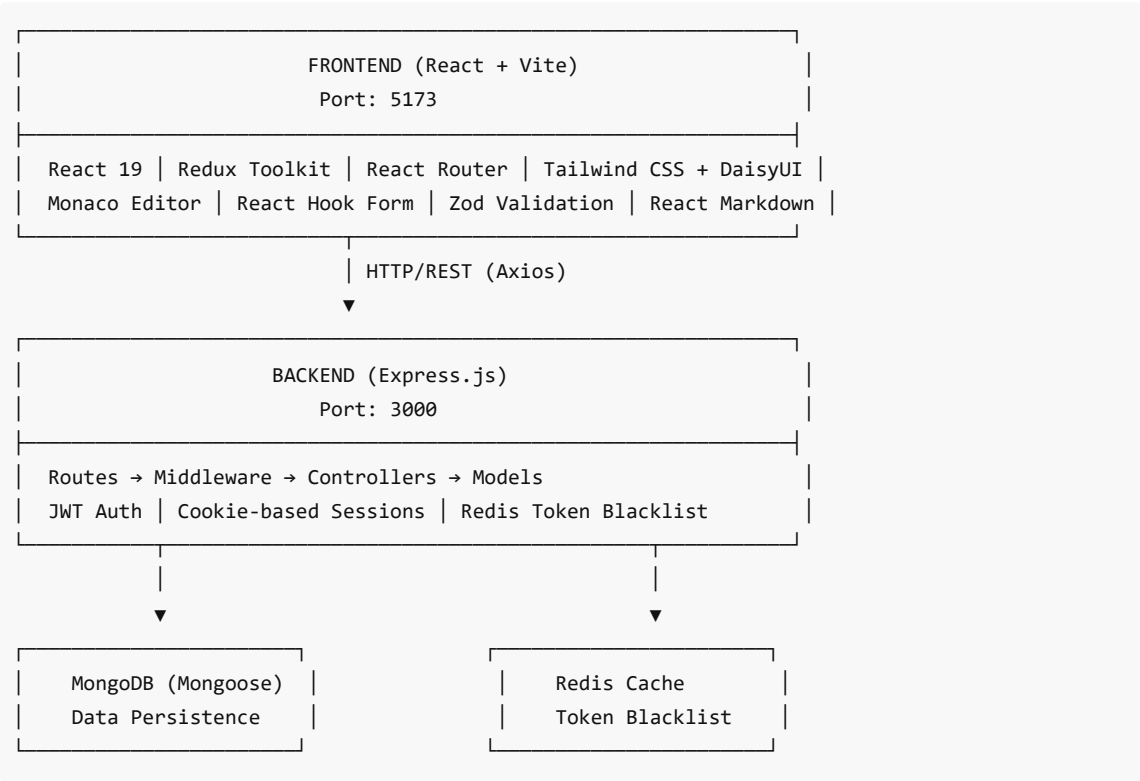


Project Architecture Overview

🏗️ High-Level Architecture

Your project follows a **3-tier architecture** with a React frontend and Node.js/Express backend.



📁 Backend Structure

Location: Backend/src/

Entry Point

- `index.js` - Express server initialization, route mounting, DB/Redis connection

Routes (API Endpoints)

Route	Path	Purpose
userAuth.js	/user	Authentication (register, login, logout)
problemCreator.js	/problem	Problem CRUD operations
submit.js	/submission	Code submission handling
aiChatting.js	/ai	AI chat/doubt solving
careerRoutes.js	/career	Career guidance AI
interviewRoutes.js	/interview	AI interview simulation

adminRoutes.js	/admin	Admin operations
videoCreator.js	/video	Solution video management
userDashboard.js	/dashboard	User dashboard data

Models (MongoDB Schemas)

User Model (models/user.js)

```
{
  firstName: String,           // required, 3-20 chars
  lastName: String,           // 3-20 chars
  emailId: String,            // required, unique, lowercase
  age: Number,                // 6-80
  role: String,               // "user" | "admin", default: "user"
  problemSolved: [ObjectId], // ref: "Problem"
  password: String,          // required, hashed
  timestamps: true
}
```

Problem Model (models/problem.js)

```
{
  title: String,              // required
  description: String,        // required
  difficulty: String,         // "easy" | "medium" | "hard"
  tags: String,              // enum: array, linkedlist, graph, dp, etc.
  visibleTestCases: [{
    input: String,
    output: String,
    explanation: String
  }],
  hiddenTestCases: [{
    input: String,
    output: String
  }],
  startCode: [{
    language: String,
    initialCode: String
  }],
  referenceSolution: [{
    language: String,
    completeCode: String
  }],
  problemCreator: ObjectId // ref: "User"
}
```

Submission Model (models/submission.js)

```

{
  userId: ObjectId,          // ref: "User"
  problemId: ObjectId,       // ref: "Problem"
  code: String,              // submitted code
  language: String,          // "javascript" | "c++" | "java"
  status: String,            // "pending" | "accepted" | "wrong" | "error"
  runtime: Number,           // milliseconds
  memory: Number,            // kB
  errorMessage: String,
  testCasesPassed: Number,
  testCasesTotal: Number,
  timestamps: true
}

```

Middleware

userMiddleware.js

- Validates JWT token from cookies
- Checks Redis blacklist for logged-out tokens
- Attaches user to `req.user`
- Returns 401 if unauthorized

adminMiddleware.js

- Verifies user role is "admin"
- Used for protecting admin routes

Controllers

Controller	Functions	Purpose
userAuthenticate.js	register, login, logout, adminRegister, deleteProfile	User authentication
userProblem.js	createProblem, updateProblem, deleteProblem, getProblemById, getAllProblem, solvedAllProblembyUser, submittedProblem	Problem management
userSubmission.js	Code execution and submission handling	
solveDoubt.js	AI-powered doubt solving	
careerGuidance.js	Career guidance AI features	
aiInterview.js	AI interview simulation	
adminController.js	Admin dashboard operations	
videoSection.js	Video upload/management with Cloudinary	

Frontend Structure

Location: Frontend/src/

Entry Point

- `main.jsx` - React root with Redux Provider and BrowserRouter

State Management

Redux Store (`store/store.js`)

```
import { configureStore } from '@reduxjs/toolkit';
import authReducer from '../authSlice';

export const store = configureStore({
  reducer: { auth: authReducer }
});
```

Auth Slice (`authSlice.js`)

- **State:** { `user`, `isAuthenticated`, `loading` }
- **Actions:** `checkAuth`, `login`, `logout`, `signup`
- **Async Thunks:** API calls to backend

Pages

Page	Route	Purpose
HomePage.jsx	/	Landing page with problem list
Login.jsx	/login	User login
SignUp.jsx	/signup	User registration
UserDashboard.jsx	/dashboard	User profile, solved problems, stats
ProblemPage.jsx	/problem/:problemId	Problem view with code editor
RoadmapGenerator.jsx	/roadmap	AI-powered learning roadmap
CareerGuidance.jsx	/career	Career advice AI
AIInterview.jsx	/ai-interview	Mock interview simulation
Admin.jsx	/admin	Admin panel entry
AdminDashboard.jsx	/admin/dashboard	Admin statistics

Components

Component	Purpose
AdminPannel.jsx	Problem creation form
AdminUpdate.jsx	Problem update form
AdminDelete.jsx	Problem deletion interface
AdminVideo.jsx	Video management

AdminUpload.jsx	File upload handling
Editorial.jsx	Problem editorial display
SubmissionHistory.jsx	Submission history view
ChatAi.jsx	AI chat component

Algorithm Visualizer Module

Location: visualizer/

A comprehensive algorithm visualization module with:

Sorting Algorithms

- Bubble Sort
- Quick Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Heap Sort

Searching Algorithms

- Binary Search
- Linear Search

Dynamic Programming

- 0/1 Knapsack
- Longest Common Subsequence (LCS)
- Longest Increasing Subsequence (LIS)
- Coin Change
- Edit Distance

Graphs

- BFS Traversal

Trees

- Binary Tree BFS

String Algorithms

- KMP Pattern Matching
- Rabin-Karp Algorithm
- Z-Algorithm
- Manacher's Algorithm
- Sliding Window
- Two Pointers

Recursion

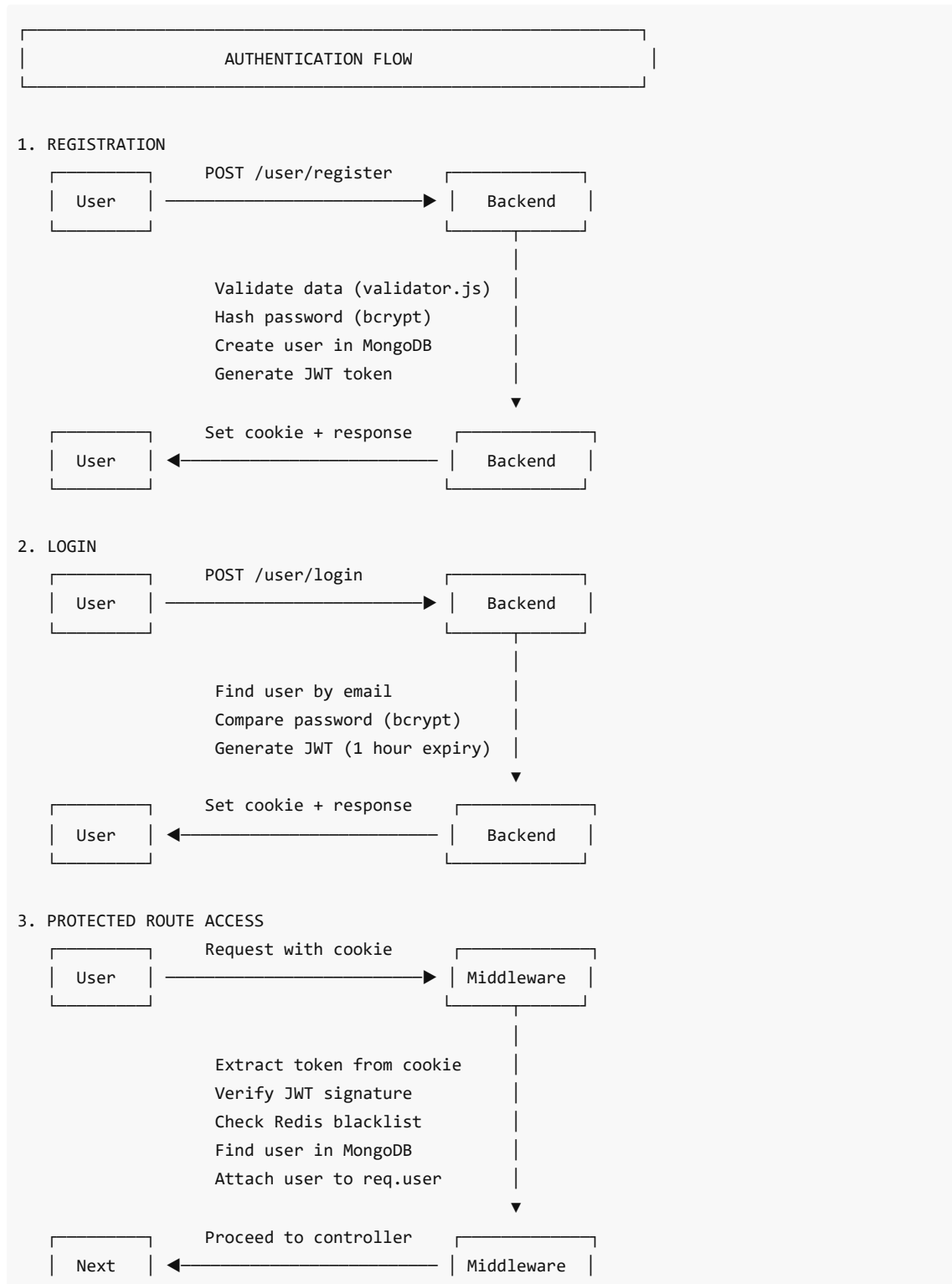
- N-Queens Problem
- Sudoku Solver
- Rat in a Maze
- Recursion Tree Visualization

Data Structures

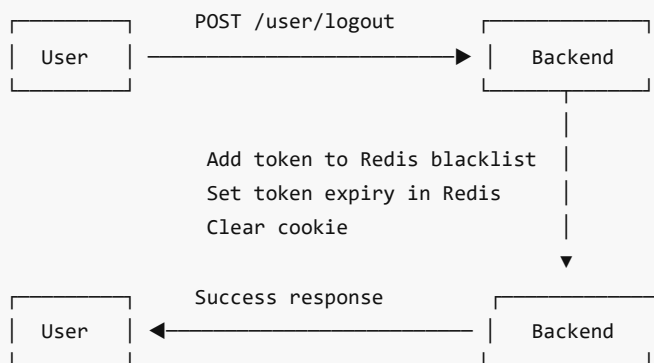
- Stack Visualization
- Queue Visualization



Authentication Flow



4. LOGOUT



Data Flow: Problem Submission

PROBLEM SUBMISSION FLOW

1. User writes code in Monaco Editor
↓
2. Frontend sends `POST /submission/submit`

```
{
  problemId: "abc123",
  code: "function solve() {...}",
  language: "javascript"
}
```


↓
3. `userMiddleware` validates JWT
↓
4. `userSubmission` controller processes request
↓
5. Code sent to Judge0 API for execution
 - Submit batch with all test cases
 - Get submission tokens
 - Poll for results
↓
6. Results stored in Submission model

```
{
  userId: ObjectId,
  problemId: ObjectId,
  code: "...",
  language: "javascript",
```

```
status: "accepted" | "wrong" | "error",
runtime: 150,
memory: 10240,
testCasesPassed: 5,
testCasesTotal: 5
}
```



7. If accepted: Update User.problemSolved array



8. Response sent to frontend with results



Key Technologies

Frontend Stack

Technology	Version	Purpose
React	19.2.0	UI library
Redux Toolkit	2.11.1	State management
React Router	7.10.1	Client-side routing
Tailwind CSS	4.1.17	Utility-first CSS
DaisyUI	5.5.8	UI component library
Monaco Editor	0.55.1	Code editor
React Hook Form	7.68.0	Form handling
Zod	4.1.13	Schema validation
Axios	1.13.2	HTTP client
React Markdown	10.1.0	Markdown rendering
Lucide React	0.560.0	Icon library
Vite	7.2.4	Build tool

Backend Stack

Technology	Version	Purpose
Express.js	5.1.0	Web framework
Mongoose	9.0.0	MongoDB ODM
JWT	9.0.2	Authentication tokens
bcrypt	6.0.0	Password hashing

Redis	5.10.0	Token blacklist cache
Google Generative AI	0.24.1	AI features
OpenAI	6.10.0	AI features
Cloudinary	2.8.0	Media storage
CORS	2.8.5	Cross-origin support
dotenv	17.2.3	Environment variables

External Services

Service	Purpose
MongoDB	Primary database
Redis	Token blacklist, caching
Judge0 API	Code execution engine
Cloudinary	Video/image storage
Google Gemini	AI features
OpenAI	AI features

Complete Directory Structure

```
d:/leetcode/
|
├── Backend/
|   ├── .env                # Environment variables
|   ├── package.json        # Dependencies
|   ├── 100_problems.json   # Problem seed data
|   └── src/
|       ├── index.js        # Server entry point
|       ├── config/
|       │   ├── db.js       # MongoDB connection
|       │   └── redis.js    # Redis client config
|       ├── controllers/
|       │   ├── adminController.js
|       │   ├── aiInterview.js
|       │   ├── careerGuidance.js
|       │   ├── solveDoubt.js
|       │   ├── userAuthenticate.js
|       │   ├── userDashboardController.js
|       │   ├── userProblem.js
|       │   ├── userSubmission.js
|       │   └── videoSection.js
|       ├── middleware/
|       └── adminMiddleware.js
```

```

|   |   └─ userMiddleware.js
|   └─ models/
|       │ └─ problem.js
|       │ └─ solutionVideo.js
|       │ └─ submission.js
|       └─ user.js
|   └─ routes/
|       │ └─ adminRoutes.js
|       │ └─ aiChatting.js
|       │ └─ careerRoutes.js
|       │ └─ interviewRoutes.js
|       │ └─ problemCreator.js
|       │ └─ submit.js
|       │ └─ userAuth.js
|       │ └─ userDashboard.js
|       └─ videoCreator.js
|   └─ utils/
|       │ └─ problemUtility.js
|       └─ validator.js
|
└─ Frontend/
    │ └─ package.json
    │ └─ vite.config.js
    │ └─ index.html
    └─ src/
        │ └─ main.jsx          # React entry point
        │ └─ App.jsx           # Root component with routes
        │ └─ authSlice.js       # Redux auth slice
        │ └─ index.css          # Global styles
        │ └─ store/
        │     │ └─ store.js      # Redux store config
        │ └─ utils/
        │     │ └─ axiosClient.js # Axios instance
        │ └─ pages/
        │     │ └─ HomePage.jsx
        │     │ └─ Login.jsx
        │     │ └─ SignUp.jsx
        │     │ └─ UserDashboard.jsx
        │     │ └─ ProblemPage.jsx
        │     │ └─ RoadmapGenerator.jsx
        │     │ └─ CareerGuidance.jsx
        │     │ └─ AIInterview.jsx
        │     └─ Admin.jsx
        │ └─ components/
        │     │ └─ AdminPannel.jsx
        │     │ └─ AdminUpdate.jsx
        │     │ └─ AdminDelete.jsx
        │     │ └─ AdminVideo.jsx
        │     │ └─ AdminUpload.jsx
        │     │ └─ Editorial.jsx
        │     │ └─ SubmissionHistory.jsx
        │     └─ ChatAi.jsx

```

```

├── visualizer/
│   ├── pages/          # Category pages
│   ├── components/     # Visualization components
│   │   ├── sorting/
│   │   ├── searching/
│   │   ├── dp/
│   │   ├── graphs/
│   │   ├── trees/
│   │   ├── strings/
│   │   ├── recursion/
│   │   └── stacks-queues/
│   ├── algorithms/     # Algorithm logic
│   └── hooks/          # Custom hooks
└── plans/
    └── roadmap-generator-ui-enhancement-plan.md

```

API Endpoints Summary

Authentication (/user)

Method	Endpoint	Description
POST	/register	Register new user
POST	/login	User login
POST	/logout	User logout
POST	/admin/register	Admin registration
DELETE	/profile	Delete user profile

Problems (/problem)

Method	Endpoint	Description
POST	/create	Create new problem (admin)
PUT	/update/:id	Update problem (admin)
DELETE	/delete/:id	Delete problem (admin)
GET	/:id	Get problem by ID
GET	/all	Get all problems
GET	/solved	Get user's solved problems
GET	/submitted/:pid	Get submissions for problem

Submission (/submission)

Method	Endpoint	Description
--------	----------	-------------

POST	/submit	Submit code for execution
------	---------	---------------------------

AI Features (/ai , /career , /interview)

Method	Endpoint	Description
POST	/ai/chat	AI doubt solving
POST	/career/guidance	Career guidance
POST	/interview/start	Start AI interview
POST	/interview/answer	Submit interview answer

Admin (/admin)

Method	Endpoint	Description
GET	/stats	Get admin statistics

Video (/video)

Method	Endpoint	Description
POST	/upload	Upload solution video
GET	/:problemId	Get video for problem

Dashboard (/dashboard)

Method	Endpoint	Description
GET	/stats	Get user statistics

Key Features

1. User Authentication

- JWT-based authentication
- Cookie-based session management
- Redis token blacklist for secure logout
- Role-based access control (user/admin)

2. Problem Management

- CRUD operations for problems
- Visible and hidden test cases
- Multiple language support (JavaScript, C++, Java)
- Starter code templates

3. Code Execution

- Integration with Judge0 API
- Real-time code execution

- Test case validation
- Runtime and memory tracking

4. AI Features

- Doubt solving assistant
- Career guidance
- AI-powered mock interviews
- Learning roadmap generation

5. Algorithm Visualizer

- Interactive visualizations
- Step-by-step execution
- Multiple algorithm categories
- Educational code panels

6. Admin Dashboard

- Problem management
- Video solution uploads
- User statistics
- Content moderation

Environment Variables

Backend (.env)

```
PORT=3000
DB_CONNECT_STRING=mongodb://...
JWT_KEY=your_jwt_secret
REDIS_HOST=...
REDIS_PORT=...
REDIS_PASS=...
```